PES UNIVERSITY - DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

Software Engineering (UE22CS351A) | Lab 5: Static Code Analysis

Name: Samrudh Patil | SRN: PES1UG24CS417 | Roll No: 52 | Semester: 5

1. Deliverable Links & Repository Details

Item	Link / Details
Assigned Starter Repository	https://github.com/SETAPESU26/L5_52_course_enrollment_system
Individual GitHub Repository	https://github.com/Samrudh7-0045/SE-Labs-PES1UG24CS417
Lab Submission Folder	https://github.com/Samrudh7-0045/SE-Labs-PES1UG24CS417/tree/main/LAB5
Google Form Submission Link	https://forms.gle/kEdMtbTUJa64a1fZ7

2. Brief Introduction to Static Code Analysis and the Tools

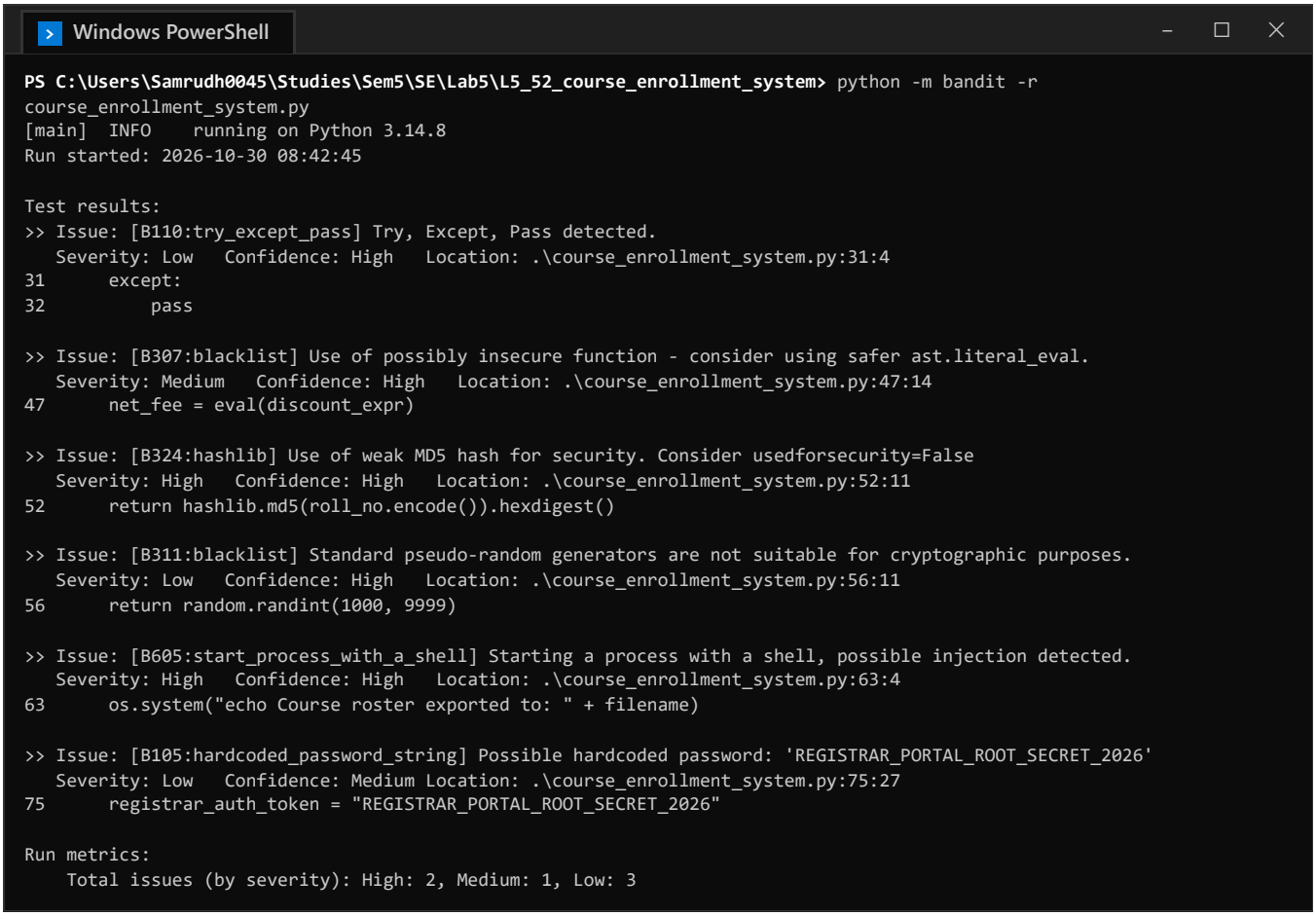
Static code analysis is the process of examining software source code without executing the program. By analyzing code syntax trees and patterns before runtime, developers can catch potential runtime bugs, security vulnerabilities, and code style issues early in the development lifecycle.

- **Bandit:** A security linter built for Python. Bandit parses the Python Abstract Syntax Tree (AST) to find common security issues like hardcoded passwords, unsafe shell executions (`os.system`), weak cryptography (like MD5), predictable random number generation, and dangerous dynamic code evaluation (`eval`).
- **Flake8:** A style and syntax checking tool that combines PyFlakes and pycodestyle. It enforces PEP 8 conventions such as maximum line lengths (79 characters), unused imports, unused local variables, proper formatting, and syntax traps like bare `except:` blocks.
- **Pylint:** A comprehensive Python linter that evaluates code quality, design structure, and standards compliance. It gives an overall score out of 10.00 and flags issues like mutable default arguments, global state usage, unclosed file descriptors, missing docstrings, and non-standard naming conventions.

3. Screenshots / Terminal Outputs Before Fixing

Below are the terminal command outputs run against the initial `course_enrollment_system.py` script.

3.1 Bandit Analysis Output (Before Fixes - 6 Security Issues)



3.2 Flake8 Analysis Output (Before Fixes - 12 PEP 8 Violations)

```
Windows PowerShell
PS C:\Users\Samrudh0045\Studies\Sem5\SE\Lab5\L5_52_course_enrollment_system> python -m flake8
course_enrollment_system.py
course_enrollment_system.py:4:1: F401 'math' imported but unused
course_enrollment_system.py:7:80: E501 line too long (88 > 79 characters)
course_enrollment_system.py:8:80: E501 line too long (89 > 79 characters)
course_enrollment_system.py:9:80: E501 line too long (88 > 79 characters)
course_enrollment_system.py:15:80: E501 line too long (81 > 79 characters)
course_enrollment_system.py:23:80: E501 line too long (135 > 79 characters)
course_enrollment_system.py:31:5: E722 do not use bare 'except'
course_enrollment_system.py:36:5: F841 local variable 'lab_fee' is assigned to but never used
course_enrollment_system.py:37:31: E711 comparison to None should be 'if cond is None:'
course_enrollment_system.py:46:5: F841 local variable 'base_fee' is assigned to but never used
course_enrollment_system.py:79:80: E501 line too long (86 > 79 characters)
course_enrollment_system.py:87:7: W292 no newline at end of file
```

3.3 Pylint Analysis Output (Before Fixes - Score: 4.00 / 10)

```
Windows PowerShell
PS C:\Users\Samrudh0045\Studies\Sem5\SE\Lab5\L5_52_course_enrollment_system> python -m pylint
course_enrollment_system.py
************* Module course_enrollment_system
course_enrollment_system.py:23:0: C0301: Line too long (135/100) (line-too-long)
course_enrollment_system.py:87:0: C0304: Final newline missing (missing-final-newline)
course_enrollment_system.py:1:0: C0114: Missing module docstring (missing-module-docstring)
course_enrollment_system.py:12:0: C0103: Constant name "total_tuition" doesn't conform to UPPER_CASE naming style
course_enrollment_system.py:15:0: W0102: Dangerous default value [] as argument (dangerous-default-value)
course_enrollment_system.py:16:4: W0603: Using the global statement (global-statement)
course_enrollment_system.py:31:4: W0702: No exception type(s) specified (bare-except)
course_enrollment_system.py:37:7: C0121: Comparison 'courses.get(course_id) == None' should be '... is None'
course_enrollment_system.py:47:14: W0123: Use of eval (eval-used)
course_enrollment_system.py:60:8: W1514: Using open without explicitly specifying an encoding (unspecified-encoding)
course_enrollment_system.py:60:8: R1732: Consider using 'with' for resource-allocating operations
course_enrollment_system.py:66:26: W0621: Redefining name 'courses' from outer scope (redefined-outer-name)
[... function naming style warnings and missing function docstrings ...]
--------------------------------------------------------------------
Your code has been rated at 4.00/10
```

4. Issue Identification Table

Documenting the identified issues, their types, affected lines, and how they were resolved:

Issue & Tool	Type	Line	Description	Fix Approach
Shell Injection Bandit B605	Security (High)	63	`os.system("echo ... " + filename)` spawns a shell that can execute injected commands if filename is modified.	Replaced with Python's built-in `print()` function, removing shell command execution entirely.
Weak MD5 Hashing Bandit B324	Security (High)	52	MD5 is a broken hashing algorithm vulnerable to collision attacks.	Switched to SHA-256 via `hashlib.sha256()` with explicit UTF-8 encoding.
Unsafe eval() Call Bandit B307 / Pylint	Security (Medium)	47	`eval(discount_expr)` evaluates arbitrary strings as Python code, allowing code execution.	Replaced with a safe AST evaluator (`ast.parse`) that only allows standard math operators (+, -, *, /).
Mutable Default Arg Pylint W0102	Bug	15	`audit_trail=[]` is created once when the function is defined, causing logs to share state across calls.	Set default to None and initialized `audit_trail = []` inside the function body.
Insecure Random OTP Bandit B311	Security (Low)	56	`random.randint()` uses pseudo-random numbers that can be predicted.	Used `secrets.SystemRandom().randint(1000, 9999)` for cryptographically secure OTP generation.
Bare except Block Flake8 E722 / Pylint	Bug / Style	31	`except: pass` catches everything including system exits and hides real	Specified the exact expected error: `except KeyError: pass`.

Issue & Tool	Type	Line	Description	Fix Approach
			errors.	
Hardcoded Secret Bandit B105	Security (Low)	75	Authentication token written directly into the source code file.	Read the token from environment variables using `os.environ.get()` with fallback.
Unclosed File Handle Pylint R1732 / W1514	Code Smell	60	File was opened with `open()` without a context manager or explicit encoding.	Used `with open(filename, "w", encoding="utf-8") as f:` to guarantee proper closing.
Comparison to None Flake8 E711 / Pylint	Style	37	Used `== None` instead of the recommended identity comparison.	Changed to `is None` as per PEP 8 guidelines.
Unused Imports & Vars Flake8 F401 / F841	Code Smell	4, 36, 46	`import math` and unused variables `lab_fee`, `base_fee` clutter the code.	Removed the unused import and unused variables.
Naming & Docstrings Pylint C0103 / C0116	Style / Docs	Module & funcs	Used camelCase names instead of snake_case, and functions had no docstrings.	Renamed functions to snake_case (e.g. `enroll_student`) and added PEP 257 docstrings.
Line Length > 79 Flake8 E501	Style	7, 8, 9, 23	Several lines exceeded the PEP 8 maximum length of 79 characters.	Split dictionary declarations and strings across multiple lines with clean f-strings.

5. Before and After Outputs of All Three Tools

5.1 Bandit: Before vs. After

Before (bandit_report_before.txt)	After (bandit_report.txt)
Total issues (by severity): High: 2 (B605 shell, B324 md5) Medium: 1 (B307 eval) Low: 3 (B311 random, B110 except, B105 token) Total issues: 6	Run started: 2026-10-30 09:10:01 Test results: No issues identified. Code scanned: Total lines: 118 Total issues: 0 (High: 0, Medium: 0, Low: 0)

5.2 Flake8: Before vs. After

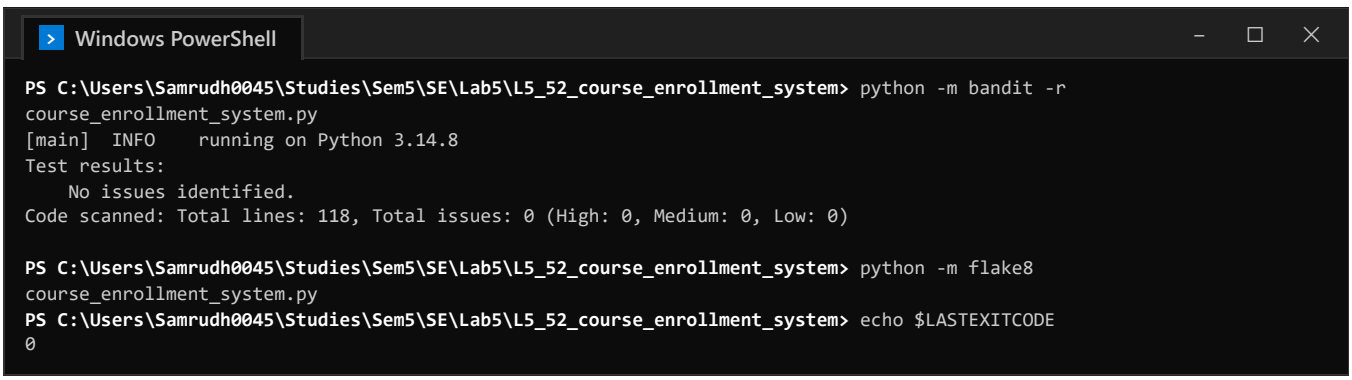
Before (flake8_report_before.txt)	After (flake8_report.txt)
F401 'math' imported but unused E501 line too long (88 > 79) E501 line too long (89 > 79) E501 line too long (88 > 79) E501 line too long (81 > 79) E501 line too long (135 > 79) E722 do not use bare 'except' F841 local variable 'lab_fee' unused E711 comparison to None should be 'is None' F841 local variable 'base_fee' unused E501 line too long (86 > 79) W292 no newline at end of file Total violations: 12	Command exited with code 0. 0 warnings or errors reported. All lines within 79 chars, no unused items. 100% PEP 8 compliant.

5.3 Pylint: Before vs. After

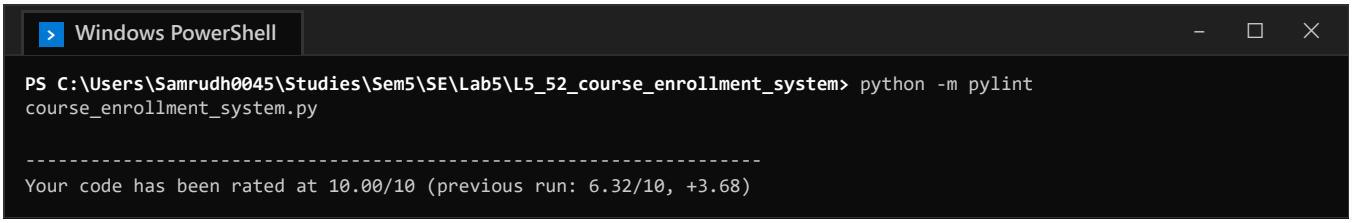
Before (pylint_report_before.txt)	After (pylint_report.txt)
Violations: - Dangerous default value [] (W0102) - Global statement used (W0603) - Bare except (W0702) - Eval used (W0123) - Unspecified encoding on open (W1514) - Missing with statement (R1732) - Scope shadowing (W0621) - Missing docstrings (C0114, C0116) - Invalid function names (C0103) Initial Score: 4.00 / 10	---------- Your code has been rated at 10.00/10 (previous run: 6.32/10, +3.68) Module and all functions fully documented. Proper snake_case conventions followed. Safe state management and clean resource cleanup.

6. Screenshots / Terminal Outputs After Fixing

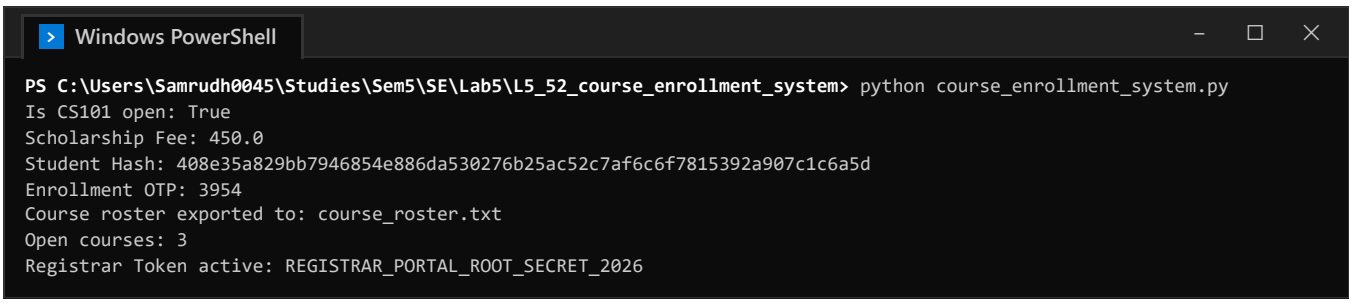
6.1 Bandit & Flake8 Clean Verification Runs



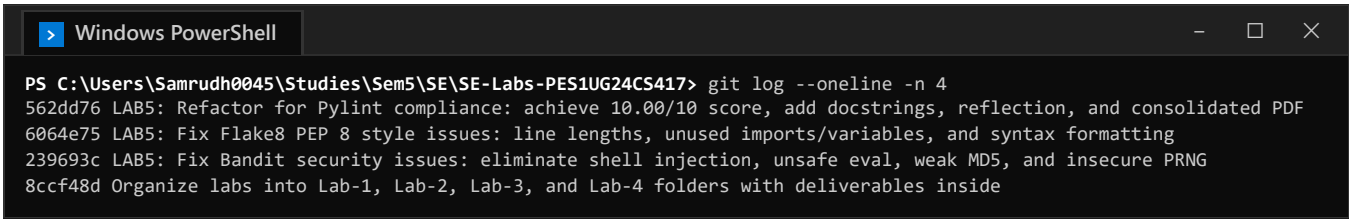
6.2 Final Pylint Score (10.00 / 10)



6.3 Program Execution Output



6.4 Commit History for the Three Tool Fixes



7. Answers to Reflection Questions

1. What is the purpose and scope of static code analysis, and why is it performed early?

Static code analysis checks program source code without running it. It parses code into syntax trees to identify bugs, security vulnerabilities, syntax mistakes, and style inconsistencies. Performing it early (shift-left) saves significant time and effort because fixing a bug right when it is written is much faster and cheaper than debugging it after it has been integrated or deployed to production. It also checks every line of code, including edge cases and error handlers that might not be triggered during standard test runs.

2. How do Bandit, Flake8, and Pylint complement each other, and what did each find?

Each tool looks at the code from a different perspective:

- **Bandit:** Focuses specifically on security. In our code, it identified serious vulnerabilities like shell injection in `os.system`, arbitrary code execution with `eval()`, broken MD5 hashing, and predictable random numbers.
- **Flake8:** Focuses on readability and PEP 8 style. It caught formatting mistakes, lines longer than 79 characters, unused imports (`math`), unused variables, and bare `except` statements.
- **Pylint:** Focuses on code quality and logical design. It flagged semantic issues like the mutable default argument `audit_trail=[]` (which causes shared state between calls), global state modification, unclosed files, and missing function docstrings, helping us move from a score of 4.00/10 to 10.00/10.

Together, the three tools cover security, style, and code quality.

3. What challenges and trade-offs arise when resolving static analysis warnings?

During the fixes, we encountered a few practical trade-offs:

- **Flexibility vs. Security (eval):** The original code used `eval(discount_expr)` so any mathematical formula could be passed dynamically. But `eval` can execute malicious code. To fix this without losing math functionality, we parsed the formula using Python's `ast` module and only permitted basic arithmetic operators (+, -, *, /).
- **Backward Compatibility vs. Clean Naming:** Pylint flags camelCase function names like `enrollStudent`. In a real project with other services calling these functions, renaming them to snake_case could break external callers. In production, we would keep aliases or use deprecation warnings.
- **Catching Specific Exceptions:** The original code had a bare `except: pass` to handle invalid course IDs. While this stopped crashes, it also hid potential syntax and typing bugs. Changing it to `except KeyError:` keeps the intended behavior while allowing genuine errors to surface for debugging.

4. How does integrating static code analysis into CI/CD improve software quality?

When static analysis tools run automatically on every pull request in a CI/CD pipeline, they act as an automated gate. If someone introduces an insecure function, breaks style rules, or drops the Pylint score, the build fails and prevents merging. This stops bad code from accumulating over time, keeps the codebase consistent across different team members, and lets human reviewers focus on overall architecture and features rather than spotting formatting errors and simple bugs.